Doc. no. P0002R0
Date: 2015-09-28
Project: Programming Language C++
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Remove Deprecated `operator++(bool)`

I. Table of Contents

I. Table of Contents

II. Introduction

The `++` operator for `bool` was deprecated in the original 1998 C++ standard, and it is past time to formally remove it.

III. Motivation and Scope

During the development of C++14, the Core working group chair suggested that it was time to remove this deprecated operator from the standard, and the author of this paper expressed a concern that the post-increment operator still had rare but valid uses.

In response to that concern, the function `std::exchange` was added to the C++14 standard library, but the paper to finally remove the deprecated operator did not follow. The intent of this paper is to finally complete the process started during the evolution of C++14.

When the (planned) C++17 standard is published, users will have had 20 years to prepare for this removal, significantly longer than has been allowed for previously removed deprecated features. See the references at the end of this paper for a list of other deprecated features expected to be removed from the next version of the standard.

IV. Impact On the Standard

The impact on the standard is minimal, as there is no known dependency in the library on this feature, and we must already accomodate the fact that the `--` operator has never been supported for `bool`.

V. Design Decisions

The wording follows, as closely as possible, the style of wording that is used to exempt `bool` from supporting the `--` operator.

Consideration was given to whether it would make sense to remove `bool` from the set of arithmetic types, given it no longer supports the full set of arithmetic operations. That seems to be a much larger issue in scope, and the proposed change seems no worse than the existing situation where the `--` operator is not supported. The small number of edits needed to effect this removal, compared to a much larger set of ammendents to rules that would need to be added if we decided that `bool` was no longer arithmetic, suggest that we are not yet in a position where we should consider that question.

Although *cv* qualified `bool` types are also arithmetic types, that distinction is ignored, in keeping with the wording that is being

replaced. An alternate phrasing of "non-`bool` arithmetic type" was also considered, with no clear preferance by the author, deferring final choice to the Core wording review.

VI. Technical Specifications

Make the following edits:

5.2.6 Increment and decrement [expr.post.incr]

1 The value of a postfix `++` expression is the value of its operand. [*Note:* the value obtained is a copy of the original value *â€”end note*] The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than `bool`, or a pointer to a complete object type. The value of the operand object is modified by adding 1 to it. ~~, unless the object is of type `bool`, in which case it is set to `true`. [*Note:* this use is deprecated, see Annex D. *â€”end note*]~~ The value computation of the `++` expression is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of postfix `++` is a single evaluation. [*Note:* Therefore, a function call shall not intervene between the lvalue-to-rvalue conversion and the side effect associated with any single postfix `++` operator. *â€”end note*] The result is a prvalue. The type of the result is the cv-unqualified version of the type of the operand. If the operand is a bit-field that cannot represent the incremented value, the resulting value of the bit-field is implementation-defined. See also 5.7 and 5.18.

2 The operand of postfix `--` is decremented analogously to the postfix `++` operator~~, except that the operand shall not be of type `bool`~~. [*Note:* For prefix increment and decrement, see 5.3.2. *â€” end note*]

5.3.2 Increment and decrement [expr.pre.incr]

1 The operand of prefix `++` is modified by adding 1~~, or set to true if it is bool (this use is deprecated)~~. The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than `bool`, or a pointer to a completely-defined object type.

13.6 Built-in operators [over.built]

3 For every pair *(T, VQ)*, where *T* is an arithmetic type other than *bool*, and *VQ* is either `volatile` or empty, there exist candidate operator functions of the form

```
VQ T& operator++(VQ T&);
T operator++(VQ T&, int);
```

C.4.2 Annex D: compatibility features [diff.cpp14.depr]

D.1

Change: Remove increment operator with `bool` operand [depr.incr.bool].

Rationale: Obsolete feature with occassionally surprising semantics.

Effect on original feature: A valid C++ 2014 expression utilizing the increment operator on a `bool` xvalue is ill-formed in this International Standard. Note that this might occur when the xvalue has a type given by a template parameter.

~~D.1 Increment operator with `bool` operand [depr.incr.bool]~~

~~1 The use of an operand of type `bool` with the `++` operator is deprecated (see 5.3.2 and 5.2.6).~~

VII. Acknowledgements

Bjarne Stroustrup filed the original issue against the C++14 CD, and Jeffrey Yasskin provided the `std::exchange` library function that can serve as a substitute in existing code relying on the deprecated feature.

VIII. References

- N3668 exchange() utility function, revision 3, Jeffrey Yasskin
- N4086 Removing trigraphs??!, Richard Smith
- N4168 Removing `auto_ptr`, Billy Baker
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- Core issue 809 Deprecation of the `register` keyword

- Core issue 1653 Removing deprecated increment of `bool`